



Universidad Nacional de Ingeniería

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

Desarrollo de Software

Práctica Calificada N°4: Implementación de una aplicación para reporte de mascotas perdidas, buscador de mascotas y red de cuidadores.

Alumno

Osorio Miranda, Arturo Miguel

Lima, Perú

01/07/2026

PC4 - Desarrollo de Software

Nombre : Arturo Miguel Osorio Miramela

Para esta PC explicaremos las historias de usuarios, las funcionalidades, la arquitectura, los patrones utilizados y finalmente una demo de casos de uso con capturas.

1. Historias de Usuario :

HU-01 : Reportar mascota perdida :

Como dueño de una mascota, quiero registrar una mascota perdida con sus datos principales y ubicación, para que otros usuarios puedan ayudar a encontrarla.

HU-02 : Registrar avistamiento :

Como ciudadano, quiero reportar que vi a una mascota perdida indicando especie, posible raza y ubicación, para ayudar al dueño.

HU-03 : Buscar mascotas para adopción o venta :

Como usuario, quiero buscar mascotas según intención de adopción o compra, para ver solo opciones de organizaciones o ciudades certificadas.

HU-04 : Consultar ciudadanos :

Como usuario verificado, quiero revisar ciudadanos disponibles y sus datos de contacto, para poder solicitar apoyo con mi mascota.

HU-05 : Administrar perfil de ciudadano :

Como ciudadano, quiero registrar mis restricciones de permiso, especies aceptadas y disponibilidad, para recibir solicitudes adecuadas.

2. Funcionalidades Implementadas

El sistema implementa una plataforma web para apoyo en casos de mascotas perdidas, búsqueda de mascotas y red de cuidadores. En esta práctica se priorizaron las funcionalidades principales y los patrones de diseño. Dentro de las funcionalidades principales tenemos:

2.1. Reporte de mascotas perdidas:

Se implementó un formulario que permite registrar una mascota perdida ingresando nombre, especie, raza, color, descripción y ubicación geográfica. La ubicación puede seleccionarse desde un mapa interactivo usando **Leaflet** y **OpenStreetMap**, ya que al intentar hacerlo con Google Maps, se requería insertar un método de pago al usar Javascript en la aplicación.

2.2. Registro de avistamientos

El sistema permite registrar avistamientos de mascotas indicando especie, raza aproximada, descripción y coordenadas. El usuario puede asociar el avistamiento a un reporte existente o marcar que no sabe cuál es.

2.3. Búsqueda por intención

Se implementó una búsqueda multipropósito basada en criterios textuales, ya que el alcance del proyecto no incluye procesamiento real de imágenes en la práctica calificada. El usuario selecciona una intención: adopción, venta o verificar pérdida. Según la opción elegida, el sistema muestra resultados distintos.

2.4. Catálogo de adopción y venta

Para adopción, el sistema muestra mascotas registradas por organizaciones o protectoras. Para venta, muestra mascotas ofrecidas por criaderos certificados. Cada resultado incluye datos básicos de la mascota y contacto de la organización.

2.5. Red de cuidadores

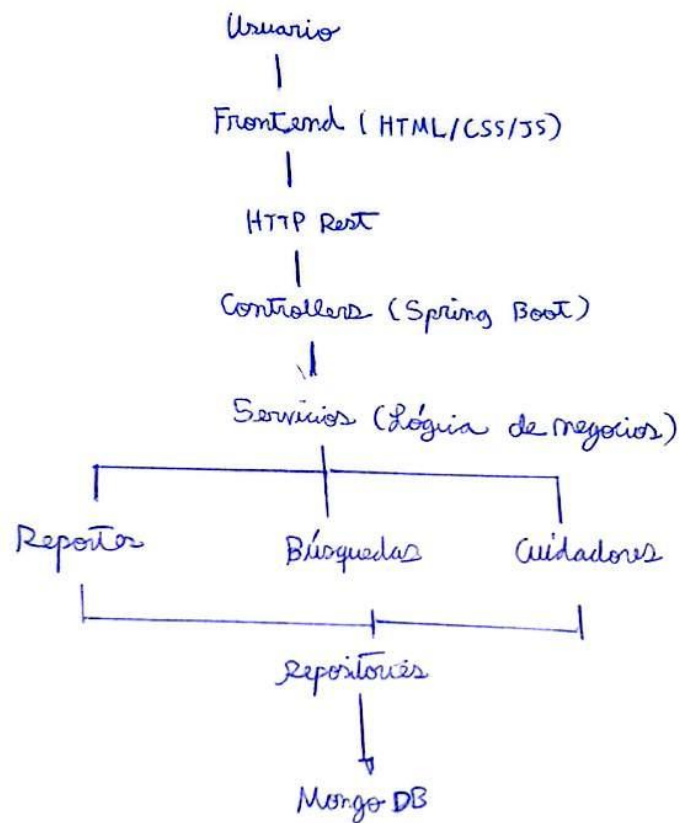
Se implementó un módulo de cuidadores con información de rol, restricciones, especies aceptadas, calificación y disponibilidad. Los datos de contacto solo son visibles para usuarios registrados y verificados mediante DNI.

2.6. Autenticación básica:

Se implementó un registro e inicio de sesión simple. El DNI de 8 dígitos se usa como validación básica para considerar al usuario como verificado.

3. Arquitectura :

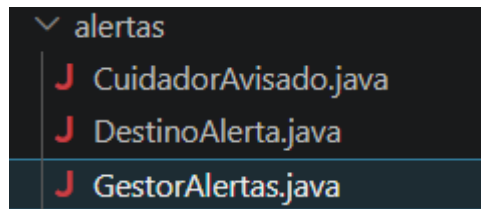
Se implementó una arquitectura de 4 capas : presentación, controlador, servicios y persistencia..



4. Patrones de Diseño Implementados

4.1. Observer

Se usa en el módulo de alertas. Cuando se registra una mascota perdida, el sistema debe avisar a los cuidadores que aceptaron recibir alertas y que se encuentran cerca del reporte. Para esto se separó la acción de registrar el reporte de la acción de avisar a los interesados.



```
J GestorAlertas.java X
backend > src > main > java > com > petalert > service > alertas > J GestorAlertas.java > {} com.petalert.service.alertas
1 package com.petalert.service.alertas;
2
3 import com.petalert.model.AvisoAlerta;
4 import com.petalert.model.ReportePerdido;
5 import org.springframework.stereotype.Component;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10 @Component
11 public class GestorAlertas {
12     public List<AvisoAlerta> notificar(ReportePerdido reporte, List<DestinoConDistancia> destinos) {
13         List<AvisoAlerta> avisos = new ArrayList<>();
14         for (DestinoConDistancia item : destinos) {
15             avisos.add(item.destino().recibirAviso(reporte, item.distanciaKm()));
16         }
17         return avisos;
18     }
19
20     public record DestinoConDistancia(DestinoAlerta destino, double distanciaKm) {
21     }
22 }
23
```

4.2. Strategy

Se aplicó el patrón Strategy en el buscador multipropósito. Según la intención seleccionada por el usuario, el sistema utiliza una forma distinta de búsqueda: adopciones, criaderos certificados o reportes de mascotas perdidas. Esto evita concentrar toda la lógica en un solo método con múltiples condicionales. Para ello tenemos varias clases denominadas BuscarEn, además existe una clase SelectorTipoBusqueda y una clase OperacionBusquedaMascota que actúa de interfaz para la operación de buscar de manera general.

▼ busqueda

- J BuscarEnAdopciones.java
- J BuscarEnCriaderos.java
- J BuscarEnPerdidos.java
- J BusquedaMascotaService.java
- J OperacionBusquedaMascota.java
- J SelectorTipoBusqueda.java

```
J SelectorTipoBusqueda.java ×
backend > src > main > java > com > petalert > service > busqueda > J SelectorTipoBusqueda.java > {} com.petalert.service.busqueda
1 package com.petalert.service.busqueda;
2
3 import com.petalert.model.IntencionBusqueda;
4 import com.petalert.service.busqueda.BuscarEnAdopciones;
5 import com.petalert.service.busqueda.BuscarEnPerdidos;
6 import com.petalert.service.busqueda.BuscarEnCriaderos;
7 import com.petalert.service.busqueda.OperacionBusquedaMascota;
8 import org.springframework.stereotype.Component;
9
10 @Component
11 public class SelectorTipoBusqueda {
12     private final BuscarEnAdopciones busquedaAdopcion;
13     private final BuscarEnCriaderos busquedaVenta;
14     private final BuscarEnPerdidos busquedaPerdidos;
15
16     public SelectorTipoBusqueda(BuscarEnAdopciones busquedaAdopcion,
17                                 BuscarEnCriaderos busquedaVenta,
18                                 BuscarEnPerdidos busquedaPerdidos) {
19         this.busquedaAdopcion = busquedaAdopcion;
20         this.busquedaVenta = busquedaVenta;
21         this.busquedaPerdidos = busquedaPerdidos;
22     }
23
24     public OperacionBusquedaMascota elegir(IntencionBusqueda intent) {
25         return switch (intent) {
26             case ADOPCION -> busquedaAdopcion;
27             case VENTA -> busquedaVenta;
28             case VERIFICAR_PERDIDA -> busquedaPerdidos;
29         };
30     }
31 }
32
```

```
J BuscarEnPerdidos.java X
backend > src > main > java > com > petalert > service > busqueda > J BuscarEnPerdidos.java > ( ) com.petalert.service.busqueda
1 package com.petalert.service.busqueda;
2
3 import com.petalert.dto.BusquedaRequest;
4 import com.petalert.dto.ResultadoBusqueda;
5 import com.petalert.model.ReportePerdido;
6 import com.petalert.model.EstadoReporte;
7 import com.petalert.repository.ReportePerdidoRepository;
8 import org.springframework.stereotype.Component;
9
10 import java.util.Comparator;
11 import java.util.List;
12
13 @Component
14 public class BuscarEnPerdidos implements OperacionBusquedaMascota {
15     private final ReportePerdidoRepository repository;
16
17     public BuscarEnPerdidos(ReportePerdidoRepository repository) {
18         this.repository = repository;
19     }
20
21     @Override
22     public String nombreFuente() {
23         return "Alertas activas";
24     }
25
26     @Override
27     public List<ResultadoBusqueda> buscar(BusquedaRequest request) {
28         return repository.findByStatus(EstadoReporte.ACTIVA).stream()
29             .filter(reporte -> coincide(reporte.getSpecies(), request.getSpecies()))
30             .filter(reporte -> coincide(reporte.getBreed(), request.getBreed()))
31             .filter(reporte -> coincide(reporte.getColor(), request.getColor()))
32             .map(this::aResultado)
33             .sorted(Comparator.comparingDouble(ResultadoBusqueda::getScore).reversed())
34             .toList();
35     }
36 }
```

4.3. Cadena de Responsabilidad

Se aplicó Cadena de Responsabilidad en la validación de cuidadores. El registro pasa por varias revisiones independientes, como documento de identidad, rol y restricciones de servicio. Esto permite agregar o cambiar validaciones sin modificar todo el proceso. La revisión entonces se realiza mediante pasos y se utiliza la operación “next”.

```
▼ validacion
J PasoRevisionCuidador.java
J RevisionCuidador.java
J RevisionDocumento.java
J RevisionRestricciones.java
J RevisionRol.java
```

```

J PasoRevisionCuidador.java X
backend > src > main > java > com > petalert > service > validacion > J PasoRevisionCuidador
1 package com.petalert.service.validacion;
2
3 import com.petalert.model.Cuidador;
4
5 public abstract class PasoRevisionCuidador {
6     private PasoRevisionCuidador next;
7
8     public PasoRevisionCuidador linkWith(PasoRevisionCuidador next)
9     {
10         this.next = next;
11         return next;
12     }
13
14     public final void validate(Cuidador cuidador) {
15         check(cuidador);
16         if (next != null) {
17             next.validate(cuidador);
18         }
19     }
20
21     protected abstract void check(Cuidador cuidador);
22 }

```

4.4. Builder

Un reporte perdido tiene varios datos: nombre, especie, raza, color, descripción, ubicación y datos del dueño. En vez de armar ese objeto directamente en el controlador, se usa una clase encargada de construirlo de manera ordenada.


```

J ArmadorReportePerdido.java X
backend > src > main > java > com > petalert > service > reporte > J ArmadorReportePerdido.java > {} com.petalert.service.reporte
1 package com.petalert.service.reporte;
2
3 import com.petalert.dto.ReportePerdidoRequest;
4 import com.petalert.model.PuntoUbicacion;
5 import com.petalert.model.ReportePerdido;
6 import com.petalert.model.EstadoReporte;
7
8 import java.time.Instant;
9
10 public class ArmadorReportePerdido {
11     private final ReportePerdido reporte = new ReportePerdido();
12
13     public ArmadorReportePerdido fromRequest(ReportePerdidoRequest request, double defaultRadiusKm) {
14         reporte.setPetName(request.getPetName());
15         reporte.setSpecies(request.getSpecies());
16         reporte.setBreed(request.getBreed());
17         reporte.setColor(request.getColor());
18         reporte.setDescription(request.getDescription());
19         reporte.setOwnerName(request.getOwnerName());
20         reporte.setOwnerContact(request.getOwnerContact());
21         reporte.setOwnerUserId(request.getOwnerUserId());
22         reporte.setAlertRadiusKm(request.getAlertRadiusKm() == null ? defaultRadiusKm : request.getAlertRadiusKm());
23         reporte.setLocation(new PuntoUbicacion(
24             request.getLocation().getLatitude(),
25             request.getLocation().getLongitude(),
26             request.getLocation().getAddressLabel()
27         ));
28         return this;
29     }
30
31     public ArmadorReportePerdido activeNow() {
32         reporte.setStatus(EstadoReporte.ACTIVA);
33         reporte.setCreatedAt(Instant.now());
34         return this;
35     }
36
37     public ReportePerdido build() {
38         return reporte;
39     }

```

4.5. Adapter

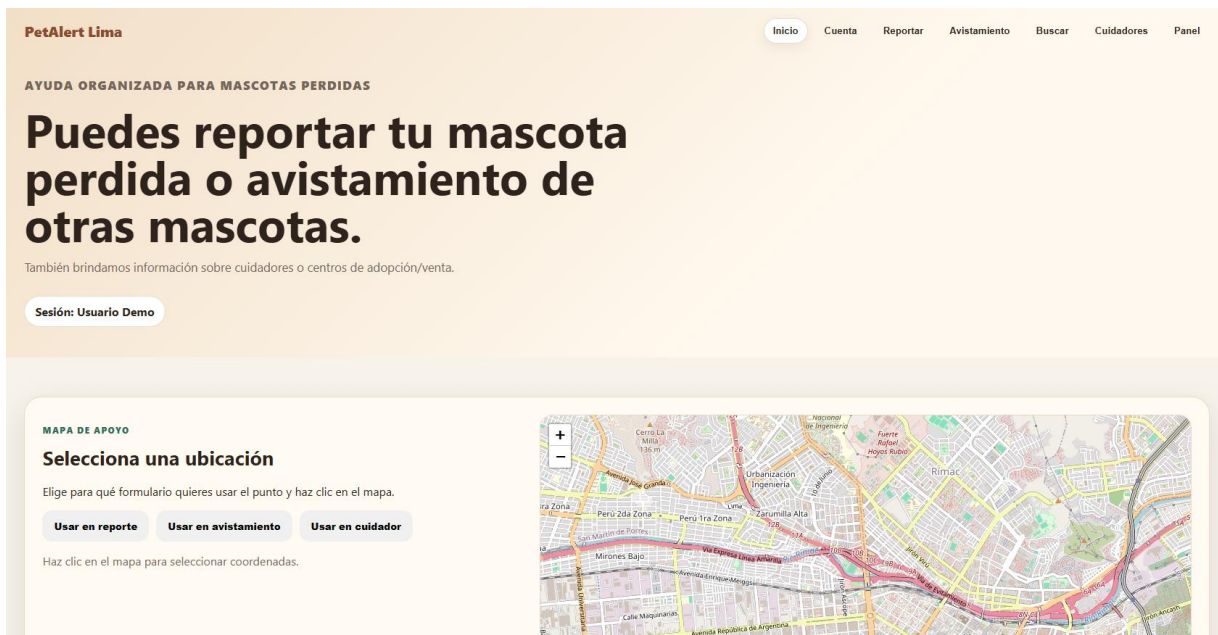
El sistema necesita trabajar con coordenadas, pero no debe depender directamente de una tecnología específica como Google Maps. Por eso la ubicación se maneja desde una clase propia del sistema. En el frontend se usa Leaflet/OpenStreetMap, pero el backend solo trabaja con latitud y longitud.

```
J UbicacionMapa.java X
backend > src > main > java > com > petalert > service > ubicacion > J UbicacionMapa.java > {} com.petalert.service.ubicacion
1 package com.petalert.service.ubicacion;
2
3 import com.petalert.model.PuntoUbicacion;
4 import org.springframework.stereotype.Component;
5
6 @Component
7 public class UbicacionMapa implements FuenteUbicacion {
8     @Override
9     public String describe(PuntoUbicacion point) {
10         if (point == null) {
11             return "Ubicacion no definida";
12         }
13         if (point.getAddressLabel() != null && !point.getAddressLabel().isBlank()) {
14             return point.getAddressLabel();
15         }
16         return "Coordenadas seleccionadas en el mapa: " + point.getLatitude() + ", " + point.getLongitude();
17     }
18 }
19
```

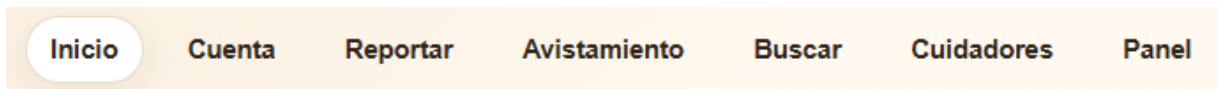
```
J FuenteUbicacion.java X
backend > src > main > java > com > petalert > service > ubicacion
1 package com.petalert.service.ubicacion;
2
3 import com.petalert.model.PuntoUbicacion;
4
5 public interface FuenteUbicacion {
6     String describe(PuntoUbicacion point);
7 }
8
```

5. Pruebas en la demo

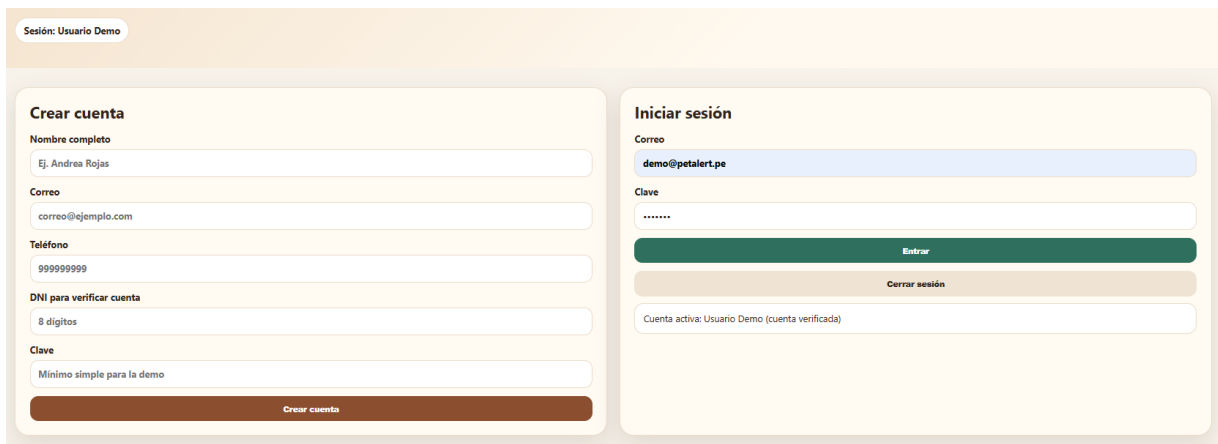
Presentamos inicialmente un panorama general del frontend:



Se cuenta con las siguientes pestañas:

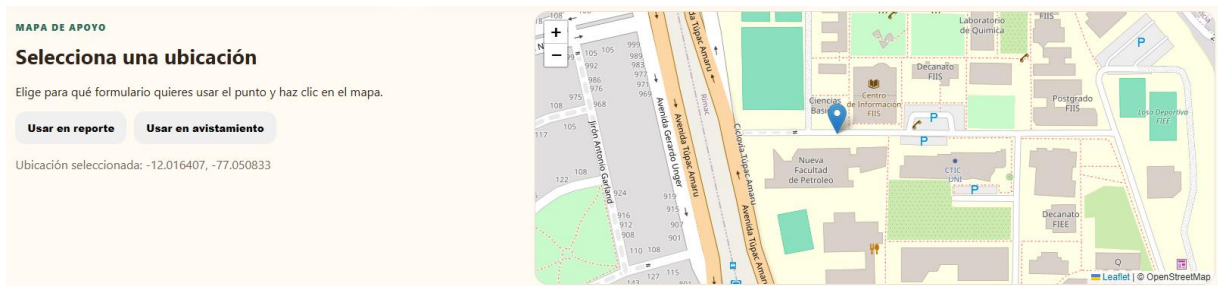


5.1. Registro e Inicio de Sesión:



5.2. Registrar Mascota Perdida siendo el dueño:

Primero se utiliza el mapa en donde se marca la ubicación en una interfaz que usa leaflet y OpenStreetMap, similar a Google Maps.



Luego se rellena el formulario con los datos:

Registrar mascota perdida

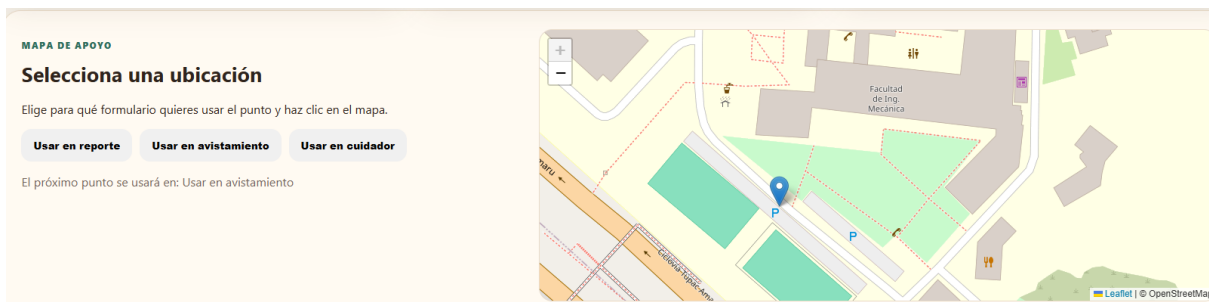
Nombre	Especie
Chise	Perro
Raza	Color
Cocker Spaniel	Marrón Claro
Descripción	
Collar rosado, conducta amigable, se vio por última vez por Cafetería FC	
Dueño	Contacto interno
E. Cóndor	957308949
Radio de alerta en km	Referencia
1	Estación Honorio Delgado
Latitud	Longitud
-12.017615	-77.050594

Guardar y disparar alerta

Al guardar se dispara la alerta y otra persona (incluido anónimo) puede reportar el avistamiento de dicha mascota.

5.3. Registrar Avistamiento de mascota para avisar al dueño:

Imaginemos que somos un usuario que se percató que una mascota está perdida. Usamos el mapa en modo avistamiento para colocar la ubicación:



Luego rellenamos el formulario. En este caso el que avistó al perro, se da cuenta de que coincide con un reporte activo, por lo que puede seleccionar el de la mascota “Chise” para avisar al usuario que la registró. Si somos lo suficiente solidarios también tenemos la opción de proporcionar un correo o un número telefónico, sin embargo, esto es opcional. Una persona que está “al paso” y “apurada” puede notificar al dueño para que se haga cargo.

Registrar avistamiento

Reporte relacionado

Chise - Perro Marrón Claro

No sé cuál es

Chise - Perro Marrón Claro

Perro

Cocker Spaniel

Color

Marrón Claro

Contacto voluntario

Celular o correo opcional

Notas

Se encontró por la Facultad de Ingeniería Mecánica, comiendo hamburguesa en jarritas que le invitaron los clientes.

Latitud

-12.024622

Longitud

-77.047485

Referencia

Altura Estación UNI, facultad de mecánica.

Enviar pista

Al finalizar la descripción se envía la pista.

5.4. Verificar mis avisos siendo el dueño de la mascota perdida

Si somos el dueño de la mascota, podemos ir al panel general y actualizar, la primera sección corresponde a avisos que estarían destinados a mi mascota perdida. Veamos:

Panel general

Actualizar

Información actualizada.

Mis avisos

Aviso recibido

Nuevo avistamiento para Chise. Referencia: Altura Estación UNI, facultad de mecánica.. Notas: Se encontró por la Facultad de Ingeniería Mecánica, comiendo hamburguesa en jarritas que le invitaron los clientes.. El ciudadano no dejó contacto.

En este caso hemos recibido notificación del avistamiento anterior.

5.5. Vista del Panel General para cualquier propósito

Cuando vamos a la pestaña de panel general al hacer click en el botón actualizar nos debe mostrar información de cuidadores y de reportes o avistamientos. Esto nos da como una visión panorámica de los datos.

Panel general

Actualizar

Información actualizada.

Mis avisos

Aún no tienes avisos.

Reportes activos

No hay reportes activos.

Avistamientos recibidos

Perro
Cocker Spaniel · Marrón Claro
Se encontró por la Facultad de Ingeniería Mecánica, comiendo hamburguesa en jarritas que le invitaron los clientes.
Reporte: No indicado
Contacto voluntario: No indicado
Referencia: Altura Estación UNI, facultad de mecánica.

Cuidadores
Los contactos se muestran solo a usuarios con cuenta verificada.

Andrea Rojas
CUIDADOR SOLIDARIO
Perro, Gato · Pequeño, Mediano
Calificación: 5.0 / 5
Alertas: activadas
Teléfono: 999111222
Correo: andrea.cuida@example.com

Carlos Vega
PROFESIONAL
Perro · Mediano, Grande
Calificación: 4.0 / 5
Alertas: activadas
Teléfono: 988777666
Correo: carlos.vega@example.com

5.6. Búsqueda Multipropósito

Solo basta con especificar si queremos adoptar o comprar en centros certificados y colocar la especie, opcional se puede considerar raza y colocar. Al buscar se desplegará la lista asociada a la base de datos. Si el usuario quiere contactar a los centros o personas encargadas, este podrá hacerlo mediante los datos que se brindan en la base de datos.

Sesión: Usuario Demo

BUSCAR MASCOTA
Adoptar, comprar o verificar pérdida

Quiero buscar para

Especie

Raza

Color

Adoptar

Perro

Opcional

Opcional

Buscar

Toby
Perro Mestizo · Marron
Sociable, vacunado y de tamaño mediano.
Edad: 1 año.
ONG: ONG Paltas Lima
Contacto: Rosa Valle · 987654321 adopciones@paltaslima.org
Contacto de la organización disponible para adopción responsable.

Nala
Perro Mestizo · Negro
Cachorra rescatada, juguetona y apta para departamento. Edad: 8 meses.
Protectora: Refugio Surco
Contacto: Miguel Campos · 956111222 contacto@refugio-surco.org
Contacto de la organización disponible para adopción responsable.

Max
Perro Labrador mestizo · Dorado
Tranquilo, esterilizado y con buen trato con niños. Edad: 2 años.
Protectora: Huellitas San Borja
Contacto: Carla Rivas · 944333222 huellitas@sanborja.org
Contacto de la organización disponible para adopción responsable.

5.7. Registro de Cuidador:

Un cuidador puede registrarse en la página para ser visualizado en la sección del panel general.

Registrar cuidador

Nombre completo

Ej. Andrea Rojas

Rol

Cuidador Solidario

DNI

8 dígitos

Teléfono de contacto

999999999

Correo de contacto

opcional@correo.com

Especies aceptadas

Perro, Gato

Tamaños aceptados

Pequeño, Mediano

☐

☒

Puede administrar medicamentos
Recibir alertas de mascotas perdidas

Latitud

-12.046374

Longitud

-77.042793

Referencia

Ej. San Isidro

Crear cuidador

5.8 Reseña de Cuidador:

Un usuario puede calificar al cuidador para que este tenga calificación y se pueda certificar que hace bien su trabajo.

Agregar reseña

Cuidador

Andrea Rojas

Tu nombre

Ej. Orlando

Calificación

5

Comentario

Buen servicio

Guardar reseña

Esto se visualiza en el panel general:

Cuidadores

Los contactos se muestran solo a usuarios con cuenta verificada.

Andrea Rojas
CUIDADOR SOLIDARIO
Perro, Gato · Pequeño, Mediano
Calificación: 5.0 / 5
Alertas: activadas
Teléfono: 999111222
Correo: andrea.cuida@example.com

Carlos Vega
PROFESIONAL
Perro · Mediano, Grande
Calificación: 4.0 / 5
Alertas: activadas
Teléfono: 988777666
Correo: carlos.vega@example.com